



Nivell 1

1. Calculadora de l'índex de massa corporal

- Escriu una funció que calculi l'IMC ingressat per l'usuari/ària, és a dir, qui ho executi haurà d'ingressar aquestes dades. Pots obtenir més informació del seu càlcul en: [Índice de masa corporal IMC que es y como se calcula](#).
- La funció ha de classificar el resultat en les seves respectives categories.

Consell: Intenta validar les dades prèviament, perquè envii un missatge d'avertència si les dades introduïdes per l'usuari/a no estan en el format adequat o no pren valors raonables.

He utilizado try, except para validar que los datos de peso sean un valor float y mayor a 0 y lo mismo para la altura, y agregue el while true para poder seguir ejecutandolo cuando da error, porque al principio si me daba error el segundo valor me hacia volver a empezar.

```
def calculoImc():
    #he colocado un while true para que en caso de error el programa no se cierre y pueda volver a ingresar los datos
    while True:
        try:
            peso = float(input("Ingrese su peso en kg, ej: 80.5 "))
            if peso <= 0:
                print("El peso no puede ser menor o igual a 0")
                continue
            break
        except ValueError:
            print("Error, ha introducido un valor invalido, intentelo nuevamente")

    while True:
        try:
            altura = float(input("Ingrese su altura en metros, ej: 1.88"))
            if altura <= 0:
                print("La altura no puede ser menor o igual a 0")
                continue
            break
        except ValueError:
            print("Error, ha introducido un valor invalido, intentelo nuevamente")

    #calculo del imc y redondeo a 2 decimales

    IMC = round(peso / (altura ** 2),2)
```

```
#funcion para determinar la categoria del imc

def categoria(IMC):
    if IMC < 18.5:
        return "Bajo peso"
    elif 18.5 <= IMC < 25:
        return "Peso normal"
    elif 25 <= IMC < 30:
        return "Sobrepeso"
    else:
        return "Obesidad"

print(f"Tu IMC es {IMC}. Clasificación: {categoria(IMC)}")

calculoImc()
```

[3] ✓ 2.2s

... Tu IMC es 21.22. Clasificación: Peso normal

2. Convertidor de temperatures

Existeixen diverses unitats de temperatura utilitzades en diferents contextos i regions. Les més comunes són Celsius (°C), Fahrenheit (°F) i Kelvin (K). També existeixen altres unitats com Rankine (°Ra) i Réaumur (°Re).

Selecciona almenys 2 conversors, de tal manera que en introduir una temperatura retorni, com a mínim, dues conversions, de manera que es puguin guardar (recorda que un print() no es pot guardar mai).

Consell: Intenta validar les dades prèviament, perquè envii un missatge d'advertència si les dades introduïdes per l'usuari/a no estan en el format adequat.

(EXTRA): Pensa una manera d'emmagatzemar totes les possibles conversions en un sol objecte (Llista? Diccionari? DataFrame?) en comptes d'escriure molts if else en funció de la temperatura d'origen i la temperatura de destí.

Creo una funcion que el usuario introduce una tempertura y su unidad, valido los datos introducidos y luego dependiendo la unidad, recorro un diccionario que contiene las conversiones a otras temperaturas, finalmente la funcion me devuelve los valores y los imprimo

```
#defino la funcion para convertir temperaturas, con validacion de datos y manejo de errores
```

```
def convertir_temperatura():
    while True:
        try:
            valor = float(input("Introduce la temperatura: "))
            break
        except ValueError:
            print(" El valor introducido no es numérico.")
    while True:
        unidad_origen = str(input("Introduce la unidad (C, F o K): ")).upper()
        if unidad_origen in ["C", "F", "K"]:
            break
        else:
            print(" Unidad no válida. Usa C, F o K.")
```

```
#CREO UN DICCIONARIO PARA LAS CONVERSIONES DE TEMPERATURA, UTILIZANDO LAMBDA PARA CALCULAR LAS CONVERSIONES DE MANERA DINAMICA
```

```
conversiones = {
    "C": {
        "F": lambda x: x * 9/5 + 32,
        "K": lambda x: x + 273.15
    },
    "F": {
        "C": lambda x: (x - 32) * 5/9,
        "K": lambda x: (x - 32) * 5/9 + 273.15
    },
    "K": {
        "C": lambda x: x - 273.15,
        "F": lambda x: (x - 273.15) * 9/5 + 32
    }
}
```

```
#hago un for para recorrer el diccionario y buscar la unidad de orgine y asignarle el resultado de la conversion
```

```
resultadosConversiones = {}
```

```
for unidad_destino, funcion in conversiones[unidad_origen].items():
    resultadosConversiones[unidad_destino] = round(funcion(valor), 2)
```

```
#la funcion devuelve los valores de la temperatura original, la unidad de origen y un diccionario con las conversiones a las otras unidades
return valor, unidad_origen, resultadosConversiones
```

```
valor, unidad_origen, resultadosConversiones = convertir_temperatura()
```

```
print("Temperatura:", valor, "°", unidad_origen,
      ": Temperatura en otras regiones", resultadosConversiones)
```

4) ✓ 0.8s

```
· Temperatura: 20.0 ° C : Temperatura en otras regiones {'F': 68.0, 'K': 293.15}
```

3. Comptador de paraules d'un text.

Escriu una funció que donat un text, mostri les vegades que apareix cada paraula. Intenta que es gestionin totes les casuístiques possibles que facin que el programa no funcioni correctament.

(EXTRA): Quina és la llargada mitjana de les paraules del text que has escrit?
“Hola com va?” hauria de retornar $(4+3+2) / 3 = 3$

Aquí creo una funció on l'usuari introdueix un text, la funció divideix el text en paraules, les neteja de tots els signes de puntuació, les converteix a minúscules, etc. i després crea un diccionari per comptar les paraules amb el seu nombre. Al final, la funció imprimeix el text, les paraules i el seu nombre, i calcula la longitud mitjana de les paraules sumant la longitud de cada paraula i dividint-la pel nombre de paraules.

```
import re
#utilizo re para eliminar los signos de puntuacion y contar solo las palabras, ademas de convertir todo a minusculas para evitar contar la misma palabra con mayusculas como diferente
#creo funcion para contar las palabras
def contador_de_palabras():
    texto = input("Introduce un texto: ")
    palabras = texto.split()
    palabras = re.findall(r'\b\w+\b', texto.lower()) #divido el texto en palabras y las palabras elimino todos los signos y convierto a minusculas para evitar contar la misma palabra con
    conteo = {}

    #hago un for para recorrer la lista de palabras y contar cuantas veces aparece cada palabra, guardando el resultado en un diccionario conteo{palabra: cantidad}
    for palabra in palabras:
        if palabra in conteo:
            conteo[palabra] += 1
        else:
            conteo[palabra] = 1

    total_longitud = sum(len(palabra) for palabra in palabras) #calculo la longitud total de todas las palabras sumando la longitud de cada palabra
    longitud_media = total_longitud / len(palabras) #calculo la longitud media dividiendo la longitud total entre el numero de palabras

    #imprimo dentro de la funcion
    print("\nConteo de palabras:")
    #por cada palabra y cantidad en conteo imprimo esa palabras y las cantidades, ademas de la longitud media de las palabras al final
    for palabra, cantidad in conteo.items():
        print(f"{palabra}: {cantidad}")
    print(f"Longitud media de las palabras: {longitud_media}")

contador_de_palabras()
```

```
#imprimo dentro de la funcion
print(texto)
print("\nConteo de palabras:")
#por cada palabra y cantidad en conteo imprimo esa palabras y las cantidades, ademas de la longitud media de las palabras al final
for palabra, cantidad in conteo.items():
    print(f"{palabra}: {cantidad}")
print(f"Longitud media de las palabras: {longitud_media}")

contador_de_palabras()
```

[6] ✓ 23.7s

... Hola estoy haciendo el ejercicio del sprint 9 para la it ACADEMY!!!! vamos Argentina!!

```
Conteo de palabras:
hola: 1
estoy: 1
haciendo: 1
el: 1
ejercicio: 1
del: 1
sprint: 1
9: 1
para: 1
la: 1
it: 1
academy: 1
vamos: 1
argentina: 1
Longitud media de las palabras: 4.785714285714286
```

4. Diccionari invers (amb possibilitat de duplicats)

Resulta que el client té una enquesta molt antiga que s'emmagatzema en un diccionari i els resultats els necessita al revés, és a dir, intercanviant les claus i els valors. Els valors i claus en el diccionari original són únics; si aquest no és el cas, la funció hauria d'imprimir un missatge d'avertiment, juntament amb una llista amb els valors associats a la clau repetida.

Fixa't en el següents exemples per veure l'output esperat.

```
[5]:
reverse_dictionary({'a':1, 'b':2, 'c':3})

[5]:
{1: 'a', 2: 'b', 3: 'c'}

[7]:
reverse_dictionary({'x':'apple', 'y':'banana', 'z':'banana'})

[7]:
{'apple': 'x', 'banana': ['y', 'z']}
```

En este ejercicio, creo una funcion que toma la clave y valor del diccionario dado y lo da vuelta. Con un for recorro el diccionario y doy vuelta los valores y los coloco en el diccionario inverso, si

hay duplicados los convierto en lista los valores del diccionario inverso para poder asignarle los valores repetidos

```
#creo la funcion
def reverse_dictionary(diccionario):
    diccionario_inverso = {}
    duplicados = {}

    # recorrer el diccionario y agregar al diccionario inverso dando vuelta clave y valor y los coloco en los diccionarios creados
    for clave, valor in diccionario.items():
        if valor not in diccionario_inverso:
            diccionario_inverso[valor] = clave
        else:
            # Registrar duplicados (valor que ya existe en el diccionario inverso lo agrego a duplicados)
            if valor not in duplicados:
                duplicados[valor] = [diccionario_inverso[valor]]

            duplicados[valor].append(clave)

            # Convertir a lista los valores duplicados en el diccionario inverso
            # para poder agregar multiples claves
            if not isinstance(diccionario_inverso[valor], list):
                diccionario_inverso[valor] = [diccionario_inverso[valor]]

            diccionario_inverso[valor].append(clave)

    # Mostrar advertencia si hay duplicados
    if duplicados:
        print("⚠ Advertencia: Hay valores repetidos en el diccionario original.")
        for valor, claves in duplicados.items():
            print(f"El valor '{valor}' está asociado a las claves: {claves}")
    #la fucnion devuelve el diccionario invertido
    return diccionario_inverso

reverse_dictionary({'x': 'apple', 'y': 'banana', 'z': 'banana'})
```

✓ 0.0s

```
⚠ Advertencia: Hay valores repetidos en el diccionario original.
El valor 'banana' está asociado a las claves: ['y', 'z']

{'apple': 'x', 'banana': ['y', 'z']}
```



Nivell 2

1. Comptador i endreçador de paraules d'un text.

El client va quedar content amb el comptador de paraules, però ara vol llegir arxius TXT i que calculi la freqüència de cada paraula ordenades dins de les entrades habituals del diccionari segons la lletra amb la qual comencen, és a dir, les claus han d'anar de la A a la Z i dins de la A hem d'anar de la A la Z. Per exemple, per a l'arxiu "tu_me_quieres_blanca.txt" la sortida esperada seria:

```
{ 'a': { 'a': 3,  
  'agua': 1,  
  'al': 2,  
  'alba': 4,  
  'alcobas': 1,  
  'alimenta': 1,  
  'alma': 1,  
  'amarga': 1,  
  'azucena': 1},  
  'b': { 'baco': 1,  
  'banquete': 1,  
  'bebe': 1,  
  'blanca': 3,  
  'boca': 1,  
  'bosques': 1,  
  'buen': 1},  
  'c': { 'cabañas': 1,  
  'carnes': 2,  
  'casta': 3,  
  'cerrada': 1,  
  'con': 4,  
  'conservas': 1,  
  'copas': 1,  
  'corola': 1,  
  'corriste': 1,  
  ...
```

En este ejercicio, leo el archivo txt y lo guardo en la variable texto y lo divido en palabras. Limpio las palabras y hago un for para el conteo de las palabras. luego hago otro for que toma cada palabra, y extrae la primer letra y creo otro diccionario con la letra inicial como clave y el diccionario de conteo de palabras como valor, por lo cual con el for palabra, frecuencia in conteo.items() para cada palabara la agrego al diccionarioPalabras asignando la letra como clave y dandole el valor de conteo al cual pertenece.


```

#utilizo os para leer el archivo txt y pprint para imprimir el resultado de manera mas legible
import os
import re
from pprint import pprint
ruta = os.listdir(r"C:\Users\gabri\Desktop\Especializacion\python\string 9") #la r para las barras invertidas en la ruta

#abro el archivo , lo leo y lo guardo en la variable texto
with open("tu me quieres blanca.txt", "r", encoding="utf-8") as archivo:
    texto = archivo.read()
#separo el texto en palabras usando split
palabras = texto.split()

palabras = re.findall(r'\b\w+\b', texto.lower()) #limpio las palabras con re
diccionarioPalabras = {}
conteo = {}
#cuanto las palabras con un for y guardo el resultado en un diccionario conteo{palabra: cantidad}
for palabra in palabras:
    if palabra not in conteo:
        conteo[palabra] = 1
    else:
        conteo[palabra] += 1
conteo = dict(sorted(conteo.items()))
#aquí por cada palabra en conteo creo un diccionario con la letra inicial como clave y un diccionario con la palabra y su frecuencia como valor.
for palabra, frecuencia in conteo.items():
    letra = palabra[0]
    if letra not in diccionarioPalabras:
        diccionarioPalabras[letra] = {} #si no esta la letra la agrego y la uno con la palabra.

        diccionarioPalabras[letra][palabra] = frecuencia
    else:
        diccionarioPalabras[letra][palabra] = frecuencia

diccionarioPalabras = dict(sorted(diccionarioPalabras.items())) #ordeno el diccionario por letra para que se imprima ordenado
pprint(diccionarioPalabras) #el pprint es para imprimir mas legible y como estaba el enunciado

```

```
pprint(diccionarioPalabras) #el pprint es para imprimir mas legible y como estaba el enunciado
```

[9] ✓ 0.0s

```

... {'a': {'a': 3,
        'agua': 1,
        'al': 2,
        'alba': 4,
        'alcobas': 1,
        'alimenta': 1,
        'alma': 1,
        'amarga': 1,
        'azucena': 1},
     'b': {'baco': 1,
        'banquete': 1,
        'bebe': 1,
        'blanca': 3,
        'boca': 1,
        'bosques': 1,
        'buen': 1},
     'c': {'cabañas': 1,
        'carnes': 2,
        'casta': 3,
        'cerrada': 1,
        'con': 4,
        'conservas': 1,
        'copas': 1,
        'corola': 1,
        'corriste': 1,
        ...
        'tú': 8},
     'u': {'un': 1, 'una': 1},
     'v': {'vestido': 1, 'vete': 1, 'vive': 1},
     'v': {'v': 5}}

```


2. Conversió de tipus de dades.

El client rep una llista de dades i necessita generar dues llistes, la primera on estaran tots els elements que es van poder convertir en flotants i l'altra on estan els elements que no es van poder convertir. Exemple de la llista que rep el client:

`[ '1.3', 'one', '1e10', 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4'] ]`

```
conversion([ '1.3', 'one' , '1e10' , 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']])
```

```
([1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4],  
 ['one', 'seven', '3-1/2', 'not-a-number'])
```

Aquí el usuario me da la lista de datos, por cada elemento en la lista verifico si es un valor simple o una lista o tupla, si es así reviso cada elemento de esa lista o tupla para convertirlo y agregarlo a la lista que corresponda. Luego Imprimo.

```
#lista de datos del cliente
datos = [
    '1.3',
    'one',
    '1e10',
    'seven',
    '3-1/2',
    ('2', 1, 1.4, 'not-a-number'),
    [1, 2, '3', '3.4']
]

#creo dos listas vacias para guardar los valores convertibles a float y los no convertibles
convertibles = []
no_convertibles = []

# para cada elemento de la lista datos hago un for if para verificar primero si es list o tuple, los recorro y veo si pueden convertirse a float
for elemento in datos:

    # Si es lista o tupla → recorrer interior
    if isinstance(elemento, (list, tuple)):
        for subelemento in elemento:
            try:
                valor = float(subelemento)
                convertibles.append(valor)
            except ValueError:
                no_convertibles.append(subelemento)

    # Si es elemento simple intento convertir a float y si no se puede lo agrego a no convertibles
    else:
        try:
            valor = float(elemento)
            convertibles.append(valor)
        except ValueError:
            no_convertibles.append(elemento)
```

```
#los imprimo

print("Convertibles:")
print(convertibles)

print("\nNo convertibles:")
print(no_convertibles)
```

[10]

✓ 0.0s

...

```
Convertibles:
[1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4]

No convertibles:
['one', 'seven', '3-1/2', 'not-a-number']
```



Nivell 3

1. Generador de Contrasenyes

Explora el funcionament del mòdul random de la llibreria numpy. [🔗 Random module in NumPy](#)

En aquest punt, el client ha detectat un problema amb les contrasenyes que utilitzen els seus treballadors. Asdf1234, dates d'aniversari o similars. Per a solucionar-ho ens ha encarregat una funció de Python que generi contrasenyes més segures. La funció ha de dependre dels següents paràmetres:

-
- longitud (int): Longitud de la contrasenya
 - majuscles (bool = True): Si hi ha d'aparèixer majúscules
 - minuscules (bool = True): Si hi ha d'aparèixer minúscules
 - numeros (bool = True): Si hi ha d'aparèixer números
 - signes (bool = False): Si hi ha d'aparèixer caràcters especials (,-\$? o similars)
-

Així doncs, si executem la funció de la següent manera:

```
crear_contrasenya(10, True, True, True, True)
```

Hauríem d'obtenir un output (que hem de poder guardar) de l'estil:

```
9Er,5Vn8P$
```

Assegura't que es compleixin tots els criteris, i que aquestes contrasenyes siguin realment aleatòries.

(EXTRA) Explora com podríem fer que la funció copiés la contrasenya automàticament al porta papers de l'ordinador (com si l'haguéssim seleccionat i fet ctrl+copy).

En este ejercicio, genero una funcion con parametros por defecto. La funcion verifica los parametros y si son true realiza una accion, esta accion es por cada parametro true, primero agregar todos los caracteres posibles a la variable caracteres y luego agregar a la variable obligatorios 1 valor aleatorio de esos caracteres para crear la base de la contrasenia que es obligatoria.

Cuando tengo ambas variables, las convierto en lista y las uno para generar la contrasenia, utilizando los caracteres obligatorios creados anteriormente y completando la longitud establecida en los parametros con caracteres alatorios de la variable caracteres que anteriormente a sido rellenada con los caracteres de los parametros activados.

Luego utilizo np.random.shuffle para finalmente mezclar todos los caracteres sino me quedaba los caracteres obligatorios al principio de la contrasenia.

```
#importo numpy para generar numeros aleatorios y string para obtener los caracteres necesarios para generar la contraseña
import numpy as np
import string
#creo la funicon con los valores por defecto.
def generar_contrasena(
    longitud: int,
    mayusculas: bool = True,
    minusculas: bool = True,
    numeros: bool = True,
    signos: bool = False
):
    #verifico la longitud y que se haya activado al menos un tipo de caracter, si no se cumple alguna de estas condiciones lanzo un error
    if longitud <= 0:
        raise ValueError("La longitud debe ser mayor que 0.")

    caracteres = ""
    obligatorios = ""
    #si los valores son true, realiza el if. cada uno asigna a la variable obligatorios un caracter aleatorio y agrego a la variable caracteres todos los caracteres para luego utilizarlos
    if mayusculas:
        obligatorios += "".join(np.random.choice(list(string.ascii_uppercase), 1))
        caracteres += string.ascii_uppercase

    if minusculas:
        obligatorios += "".join(np.random.choice(list(string.ascii_lowercase), 1))
        caracteres += string.ascii_lowercase

    if numeros:
        obligatorios += "".join(np.random.choice(list(string.digits), 1))
        caracteres += string.digits

    if signos:
        obligatorios += "".join(np.random.choice(list(string.punctuation), 1))
        caracteres += string.punctuation
    if not caracteres:
        raise ValueError("Debe activarse al menos un tipo de carácter.") # si no hay ningun true en los parametros te lo solicita obligatoriamente.

    # Convertimos a lista para numpy
    caracteres_array = list(caracteres)
    obligatorios_array = list(obligatorios)
    #genero la contraseña uniendo los caracteres obligatorios y luego agregando caracteres aleatorios hasta completar la longitud deseada
    contraseña = "".join(obligatorios_array) + "".join(
        np.random.choice(caracteres_array, size=(longitud-len(obligatorios_array)))) #la longitud es la que dan en el parametro - la cantidad que hay en obligatorios.

    #para que no se vean los caracteres obligatorios al principio de la contraseña, mezclo la contraseña generada con shuffle
    contraseña_lista = list(contraseña)
    np.random.shuffle(contraseña_lista)
    contraseña = "".join(contraseña_lista)

    print(obligatorios)
    return contraseña

generar_contrasena(10,True,True,True,False)
```

47] ✓ 0.0s

On0

'Xn0H0FsjnM'

2. Processament de dades simple

Una companya de feina ens ha demanat un favor, aprofitant que sap que estem aprenent a programar. Té un històric de partits de futbol català en un fitxer, on hi ha emmagatzemat els noms dels equips i els resultats. Necessita que processem les dades de manera automàtica, per extreure'n els resultats que necessita.

Utilitza el fitxer "historic_partits.txt"

Necessita un programa que retorni:

- El nombre total de gols que ha fet cada equip.
- El nom de l'equip més golejador.
- El nom de l'equip més golejat
- La classificació global (cada victòria: 3 pts, empat 1 pts, derrota 0 pts)

Com a referència, els equips amb més punts serien els següents:

	partits guanyats	partits empatats	partits perduts	gols	gols encaixats	punts
Girona FC	31	3	13	139	94	96
Llagostera	29	7	20	159	142	94
Sabadell	26	7	15	141	121	85
Cornellà	25	7	22	147	146	82
RCD Espanyol	23	11	21	131	144	80
Figueres	23	10	23	161	154	79
Lleida Esportiu	23	6	23	129	133	75
Terrassa	20	14	23	147	164	74
FC Barcelona	22	7	15	125	115	73

Lo primero que hago es leer el archivo txt, al estar separado por lineas es facil, recorro el archivo por linea y utilizo cada linea como elemento de la lista Partidos. Ahora cada elemento de la lista partidos es un partido.

Luego creo la funcion datosequipos donde ingresaremos los datos de cada equipo, como si fuera registros de un usuario. Creando un diccionario para cada equipo con los valores de la tabla.

```

import pandas as pd
#para poder al final convertirlo a dataframe

# Ruta del archivo
ruta = r"C:\Users\gabri\Downloads\historic partits.txt"

#abrir archivo y guardarlo en "archivo" y luego crear una lista de partidos con cada línea del archivo
with open(ruta, 'r', encoding="utf-8") as archivo:
    partidos = [linea.strip() for linea in archivo]

#defino la funcion que almacenara la informacion de cada equipo
clasificacion = {}
def datosEquipos(equipo):
    return {
        "PJ": 0,
        "PG": 0,
        "PE": 0,
        "PP": 0,
        "GF": 0,
        "GC": 0,
        "PTS" : 0
    }

```

Luego recorro la lista de los Partidos, y por cada partido separo los valores que estan unidos por \t en local, resultado y visitante. y para resultado separo tambien por el "-" en goles local(primer valor) - goles visitante(segundo valor), de esa manera ya obtengo todas las variables que necesito, luego se hace un if elif para determinar los resultados de los partidos y por cada equipo local o visitante agrega al diccionario clasificacion los datos nuevos de cada equipo a medida que pasan los partidos. Utilizo los goles local y goles visitante para determinar todos los puntos. Finalmente ordeno el diccionario por el item PTS (puntos) de mayor a menor y antes de imprimirlo lo convierto a dataframe ordenado por index para que quede la tabla bonita.

```

    },
    #por cada item de la lista que es un partido, determino los valores a modificar de cada equipo participante del partido
    for partido in partidos:
        local, resultado, visitante = partido.split("\t")
        golesLocal, golesVisitante = map(int, resultado.split("-"))

        if local not in clasificacion:
            clasificacion[local] = datosEquipos(local)
        if visitante not in clasificacion:
            clasificacion[visitante] = datosEquipos(visitante)
        #partidos jugados y goles a favor y en contra
        clasificacion[local]["PJ"] += 1
        clasificacion[visitante]["PJ"] += 1
        clasificacion[local]["GF"] += golesLocal
        clasificacion[local]["GC"] += golesVisitante
        clasificacion[visitante]["GF"] += golesVisitante
        clasificacion[visitante]["GC"] += golesLocal
        #determinar puntos para cada equipo
        if golesLocal > golesVisitante:
            clasificacion[local]["PG"] += 1
            clasificacion[local]["PTS"] += 3
            clasificacion[visitante]["PP"] += 1
        elif golesLocal < golesVisitante:
            clasificacion[visitante]["PG"] += 1
            clasificacion[visitante]["PTS"] += 3
            clasificacion[local]["PP"] += 1
        else:
            clasificacion[local]["PE"] += 1
            clasificacion[local]["PTS"] += 1
            clasificacion[visitante]["PE"] += 1
            clasificacion[visitante]["PTS"] += 1

    #ordeno el diccionario por el item
    clasiordenada = dict(sorted(clasificacion.items(), key=lambda item: item[1]["PTS"], reverse=True))

    df = pd.DataFrame.from_dict(clasiordenada, orient='index')

    print(df)

```

✓ 0.0s